Reto de Automatización QA - FrontEnd

1. Objetivo

La automatización tiene como objetivo validar el flujo principal de compra de SauceDemo utilizando Playwright con Cucumber y Java. Se contemplan escenarios positivos y negativos, validación del inicio de sesión, gestión del carrito y finalización de la compra.

2. Estrategia

Los features están organizados por funcionalidad para facilitar su lectura y mantenimiento:

- login.feature - escenarios de inicio de sesión.
- products.feature - gestión de productos.
- cart.feature - validación y gestión del carrito.
- checkout.feature - proceso de compra.

Se mantiene una separación de responsabilidades entre configuración de ambientes, datos de prueba, escenarios Gherkin, step definitions, steps y page.

3. Datos de prueba

- **users.json:** centraliza las credenciales utilizadas por los escenarios de login, incluyendo usuarios válidos, bloqueados e inválidos.
- **`products.csv` y `customers.csv`** permiten mantener datos de prueba fuera del código. El lector CSV es genérico y permite localizar registros mediante una columna clave configurable, facilitando su reutilización con diferentes estructuras de archivos.

4. Validación de la interfaz

Las validaciones se realizan mediante Page Objects y assertions en la capa de Steps. Se valida la visibilidad de elementos, resultados de login, productos agregados o eliminados del carrito y la confirmación final de la compra.

Playwright utiliza auto-waiting y esperas asociadas al estado de los elementos y la navegación, evitando depender de esperas fijas durante la ejecución.

5. Patrones y principios utilizados

Se utiliza el patrón **Page Object Model (POM)** para encapsular los elementos y acciones de cada página, evitando duplicar selectores y facilitando el mantenimiento.

También se aplica separación de responsabilidades mediante la siguiente estructura:

- Feature -> Step Definitions -> Steps -> Pages

6. Ejecución

La suite completa puede ejecutarse mediante:

- *mvn clean test (Por defecto se utiliza el ambiente QA)*

Para ejecutar en local:

- *mvn clean test "-Denv=local"*

Para ejecutar un conjunto específico de escenarios:

- *mvn clean test "-Dcucumber.filter.tags=@cart" "-Denv=local"*

Los reportes HTML y JSON de Cucumber se generan dentro de target/. Cuando un escenario falla, se captura una evidencia de pantalla y se adjunta al reporte.

7. Conclusión

La estrategia busca mantener una automatización simple, reutilizable y mantenible, separando la lógica de negocio, la interacción con la interfaz, los datos de prueba y la configuración. El uso de Cucumber facilita la lectura de los escenarios y POM reduce el acoplamiento entre las pruebas y la interfaz de SauceDemo.